

Universidade Federal do Ceará
Campus Quixadá
QXD0068 - Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado

Enzo Hariel - 566785
Victor Veras - 571603

Quixadá-CE
2026

Sumário

1. Introdução
2. Caracterização e Diagnóstico do Sistema Legado
3. Padrões de Projeto Selecionados
4. Refatorações Realizadas
5. Análise dos Resultados
6. Conclusão
7. Referências

1. Introdução

Reuso de software busca aproveitar soluções já existentes e reduzir o esforço de manutenção e evolução. Em sistemas legados, a refatoração com padrões de projeto é uma das formas de alcançar esse reuso: o comportamento externo é mantido e a estrutura interna passa a ser mais clara, coesa e extensível.

O sistema escolhido foi o **Powerzone Inventory**, aplicação web de gestão de estoque, transações e entregas de uma empresa de combustíveis de Catanduanes, nas Filipinas. Foi desenvolvido em JavaScript com Node.js, Express e MongoDB (Mongoose), e tem código aberto. A escolha se deu porque o sistema tem porte pequeno, já possui uma suíte de 190 testes e concentra muita lógica duplicada nos controllers, o que o torna adequado para refatoração.

O objetivo do trabalho foi refatorar a lógica de mudança de status de transações e entregas aplicando quatro padrões GoF (Strategy, State, Template Method e Adapter), preservando o comportamento original e comprovando essa preservação por testes.

Repositório (fork com as duas versões):https://github.com/harielfr/powerzone-inventory. A versão original está na tag **v1-original** e a refatorada na tag **v2-refatorado** (branch refactor). O projeto original é https://github.com/STSWENG-T1-AY2122-AWS-CodeBuild/powerzone-inventory e seus autores são creditados no repositório.

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

O Powerzone Inventory controla três componentes do negócio: o estoque de combustíveis (lotes comprados), as transações de venda e as entregas associadas a cada transação. As principais funcionalidades são cadastro e login de contas, registro de compras de estoque, criação e edição de transações, e acompanhamento de entregas.

- **Linguagem e tecnologias:** JavaScript, Node.js, Express, Handlebars, MongoDB com Mongoose, Mocha, Chai e Sinon para testes, ESLint.
- **Estrutura original:** pastas controllers (lógica das rotas), models (esquemas e db.js, que encapsula o acesso ao banco com callbacks), routes, views, public e test.
- **Tipos de combustível:** gasolina, gasolina premium 95, diesel, gasolina premium 97 e querosene.
- **Regra de negócio central (estoque FIFO):** ao cancelar uma transação pendente ou concluída, os litros voltam ao estoque, começando pelo lote mais recente. Ao reativar uma transação cancelada, os litros são consumidos, começando pelo lote mais antigo. As demais transições não alteram o estoque.

2.2 Problemas Identificados

O foco do diagnóstico foram os seis handlers de mudança de status: postEditStatusCancelled, postEditStatusCompleted e postEditStatusPending, em transaction-controller.js e em delivery-controller.js. Os problemas encontrados foram:

- **Duplicação de código:** os três handlers de transação somam 1088 linhas quase idênticas. Cada um repete 5 vezes (uma por combustível) o mesmo bloco de ordenar lotes e consumir ou devolver litros. No total são 15 laços while, 15 ordenações e 15 gravações de estoque repetidas.
- **Condicionais excessivos:** 90 ocorrências de if nesses três handlers e uma cadeia de if/else if com o tipo do lote para separar os combustíveis. O status anterior da transação é testado com if em cada handler.
- **Responsabilidades excessivas e alto acoplamento:** cada handler, sozinho, atualiza a transação, lê o estoque, aplica a regra FIFO, grava o estoque, atualiza a entrega e responde ao cliente. Os controllers chamam db.js diretamente, com callbacks aninhados em vários níveis.
- **Dificuldade de manutenção e extensão:** incluir um novo combustível ou mudar a regra de estoque exige alterar o mesmo trecho em todos os handlers. O controller de transações tinha 2804 linhas.

Evidência: a Figura 1 mostra o handler original de cancelamento (v1-original). A parte de cima mostra as chamadas de db.js aninhadas em callbacks e o if sobre o status anterior (linha 157). A parte de baixo mostra o bloco de ordenar lotes e devolver estoque de um combustível (gasolina). Esse mesmo bloco se repete para os outros quatro combustíveis e nos outros dois handlers.

v1-original: controllers/transaction-controller.js, linhas 142 a 162

```
142  postEditStatusCancelled: function(req, res) {
143      /* Retrieve the ID and original status of the selected transaction. */
144      const transactionId = req.body.transactionId;
145      const transactionStatusOld = req.body.transactionStatusOld;
146
147      /* Set the status of the selected account to "Cancelled". */
148      const update = {status: 'cancelled'};
149
150      /* Update the corresponding database entry using its transaction ID. */
151      const filter = {id: transactionId};
152
153      db.updateOne(Transaction, filter, update, function(flag) {
154          /* If the original status of the transaction is "Pending" or "Completed", return the fuel
155           * purchased in the transaction to their respective stocks.
156           */
157          if (transactionStatusOld == 'pending' || transactionStatusOld == 'completed') {
158              /* Retrieve the fuel amounts purchased in the transaction from the database. */
159              query = {id: transactionId};
160              projection = 'id litersGasoline litersPremiumGasoline95 litersDiesel litersPremiumGasoline97 litersKerosene';
161
162              db.findOne(Transaction, query, projection, function(result) {
```

v1-original: controllers/transaction-controller.js, linhas 200 a 236

```
200  if (transaction.litersGasoline > 0) {
201      /* If there are multiple stocks for the fuel type, arrange them in chronological order. */
202      if (stocksGasoline.length >= 2) {
203          stocksGasoline.sort(function(a, b) {
204              const keyA = a.date;
205              const keyB = b.date;
206
207              if (keyA > keyB) {
208                  return 1;
209              }
210              if (keyA < keyB) {
211                  return -1;
212              }
213              return 0;
214          });
215      }
216
217      /* Return fuel quantities to the stocks, starting from the latest stock,
218       * until the original quantity requested in the transaction has been depleted.
219       */
220      let transactionQuantity = transaction.litersGasoline;
221      let i = stocksGasoline.length - 1;
222
223      while (transactionQuantity > 0) {
224          const depletedStock = stocksGasoline[i].quantityDepleted;
225
226          if (transactionQuantity >= depletedStock) {
227              transactionQuantity = transactionQuantity - depletedStock;
228              stocksGasoline[i].quantityDepleted = 0;
229          } else {
230              stocksGasoline[i].quantityDepleted = parseInt(stocksGasoline[i].quantityDepleted) - parseInt(transactionQuantity);
231              transactionQuantity = 0;
232          }
233
234          i--;
235      }
236  }
```

Figura 1: transaction-controller.js em v1-original, postEditStatusCancelled (linhas 142 a 162 e 200 a 236).

3. Padrões de Projeto Selecionados

Foram aplicados quatro padrões GoF, que se complementam na refatoração dos handlers de status.

3.1 Strategy (comportamental)

- **Problema:** a regra de consumir ou devolver estoque estava copiada 15 vezes.
- **Justificativa:** consumir (do lote mais antigo) e devolver (do lote mais recente) são duas políticas intercambiáveis sobre a mesma estrutura de dados.
- **Componentes:** ConsumeStrategy, ReturnStrategy e NoStockChangeStrategy (Null Object), em services/stock-strategies.js; fuel-types.js (registro dos cinco combustíveis); stock-service.js.
- **Resultado esperado:** cada regra escrita uma única vez, aplicada a qualquer combustível.

3.2 State (comportamental)

- **Problema:** o status anterior da transação era testado com if em cada handler para decidir se o estoque muda.

- **Justificativa:** o comportamento depende do estado atual (pendente, concluída ou cancelada), o que é o caso típico do State.
- **Componentes:** TransactionState e as subclasses PendingState, CompletedState, CancelledState e UnknownState, em services/transaction-state.js.
- **Resultado esperado:** o estado devolve a estratégia correta para a transição, eliminando os if de status.

3.3 Template Method (comportamental)

- **Problema:** seis handlers (três de transação e três de entrega) com a mesma sequência de passos e diferindo apenas no status de destino.
- **Justificativa:** o esqueleto do algoritmo é fixo (atualizar, propagar, responder) e só alguns passos variam.
- **Componentes:** StatusChangeHandler (classe base), TransactionStatusHandler, DeliveryStatusHandler e suas subclasses por status, na pasta handlers.
- **Resultado esperado:** cada handler concreto define apenas o status de destino.

3.4 Adapter (estrutural)

- **Problema:** os controllers dependiam de db.js, de interface baseada em callbacks, e repetiam o cálculo de identificadores.
- **Justificativa:** é preciso adaptar a interface existente (db.js) para a interface que o restante do código espera (métodos que retornam Promises), sem alterar db.js.
- **Componentes:** InventoryRepository, TransactionRepository, DeliveryRepository e o módulo ids.js, na pasta repositories.
- **Resultado esperado:** controllers e serviços desacoplados do formato de callbacks e do banco, e mais fáceis de testar com dublês.

4. Refatorações Realizadas

Antes de refatorar foram escritos 24 testes de caracterização, que fixam o comportamento observável dos handlers originais (ver seção 4.5). O trabalho foi dividido em branches por padrão e integrado na branch refactor por Pull Requests.

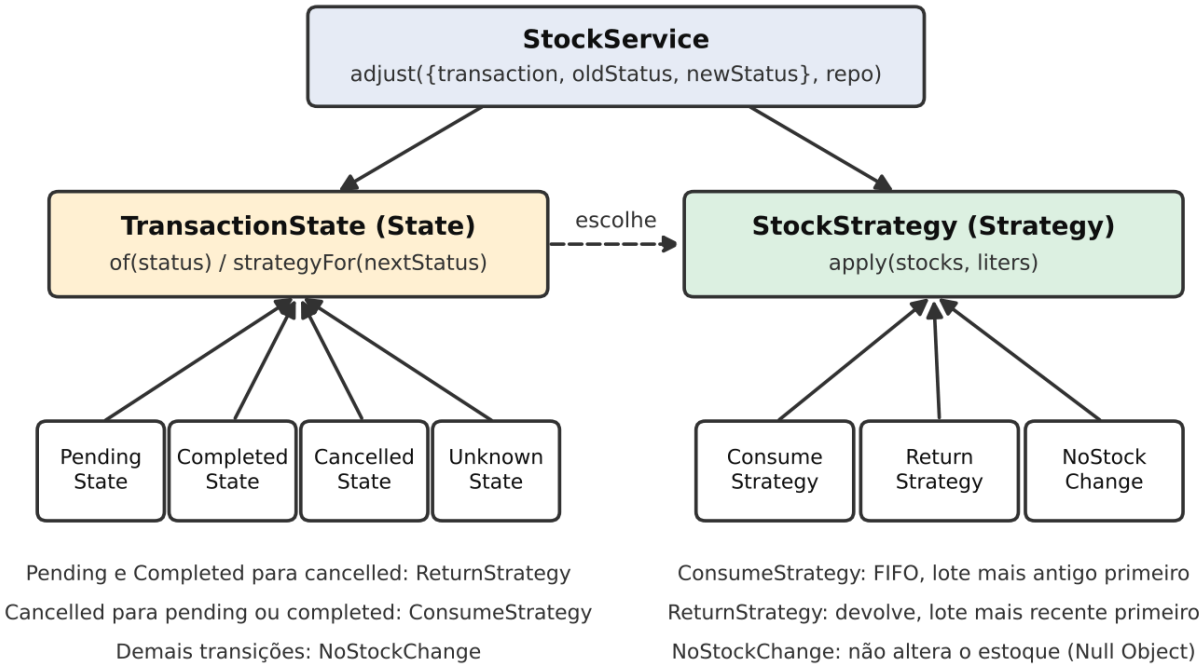


Figura 2: Strategy e State (classes e relações).

4.1 Padrão 1: Strategy

- **Situação original:** blocos de ordenar lotes e consumir ou devolver litros copiados para cada combustível e cada handler.
- **Problema:** 15 laços while e 15 ordenações iguais.
- **Solução:** duas estratégias com o método apply(stocks, liters) e um registro de combustíveis (fuel-types.js) que agrupa os lotes por tipo. O tipo desconhecido continua caindo em querosene, como no original.

O bloco original de devolução de estoque (linhas 200 a 236 da Figura 1) aparece 15 vezes. A Figura 4 mostra a versão refatorada, escrita uma única vez e aplicada a qualquer combustível.

v2-refatorado: services/stock-strategies.js, linhas 53 a 75

```
53  /* pending|completed -> cancelled: devolve o estoque a partir do lote mais recente. */
54  class ReturnStrategy {
55      apply(stocks, liters) {
56          sortByDate(stocks);
57
58          let transactionQuantity = liters;
59          let i = stocks.length - 1;
60
61          while (transactionQuantity > 0) {
62              const depletedStock = stocks[i].quantityDepleted;
63
64              if (transactionQuantity >= depletedStock) {
65                  transactionQuantity = transactionQuantity - depletedStock;
66                  stocks[i].quantityDepleted = 0;
67              } else {
68                  stocks[i].quantityDepleted = parseInt(stocks[i].quantityDepleted) - parseInt(transactionQuantity);
69                  transactionQuantity = 0;
70              }
71
72              i--;
73          }
74      }
75  }
```

Figura 4: Strategy. ReturnStrategy em stock-strategies.js (v2-refatorado). ConsumeStrategy tem a mesma estrutura, na ordem inversa.

4.2 Padrão 2: State

- **Situação original:** if (transactionStatusOld == ...) repetido nos três handlers.
- **Problema:** a regra de transição estava espalhada e misturada com o acesso ao banco.
- **Solução:** cada estado conhece a estratégia da transição. TransactionState.of(status) devolve o estado, e um status desconhecido ou ausente vira UnknownState, que não altera o estoque.

A condição original aparece na linha 157 da Figura 1. A Figura 5 mostra a versão refatorada: cada estado decide a estratégia da transição.

v2-refatorado: services/transaction-state.js, linhas 10 a 42

```
10  class TransactionState {
11      /* Por padrão, nenhuma transição mexe no estoque. */
12      strategyFor(nextStatus) {
13          return NoStockChange;
14      }
15
16      /* Devolve o estado correspondente ao status. Status desconhecido ou ausente vira o estado nulo.
17       * O original compara com ==, então um valor que se converte na string do status (ex.: um array
18       * vindo do corpo da requisição) também casa; String() reproduz essa conversão.
19       */
20      static of(status) {
21          const key = status === undefined || status === null ? status : String(status);
22          return STATES.has(key) ? STATES.get(key) : UNKNOWN_STATE;
23      }
24  }
25
26  class PendingState extends TransactionState {
27      strategyFor(nextStatus) {
28          return nextStatus === 'cancelled' ? ReturnStrategy : NoStockChange;
29      }
30  }
31
32  class CompletedState extends TransactionState {
33      strategyFor(nextStatus) {
34          return nextStatus === 'cancelled' ? ReturnStrategy : NoStockChange;
35      }
36  }
37
38  class CancelledState extends TransactionState {
39      strategyFor(nextStatus) {
40          return nextStatus === 'pending' || nextStatus === 'completed' ? ConsumeStrategy : NoStockChange;
41      }
42  }
```

Figura 5: State. transaction-state.js (v2-refatorado).

O StockService.adjust junta os dois padrões: obtém a estratégia pelo estado atual, lê os lotes pelo repositório, aplica a estratégia para cada combustível com litros maiores que zero e grava o estoque.

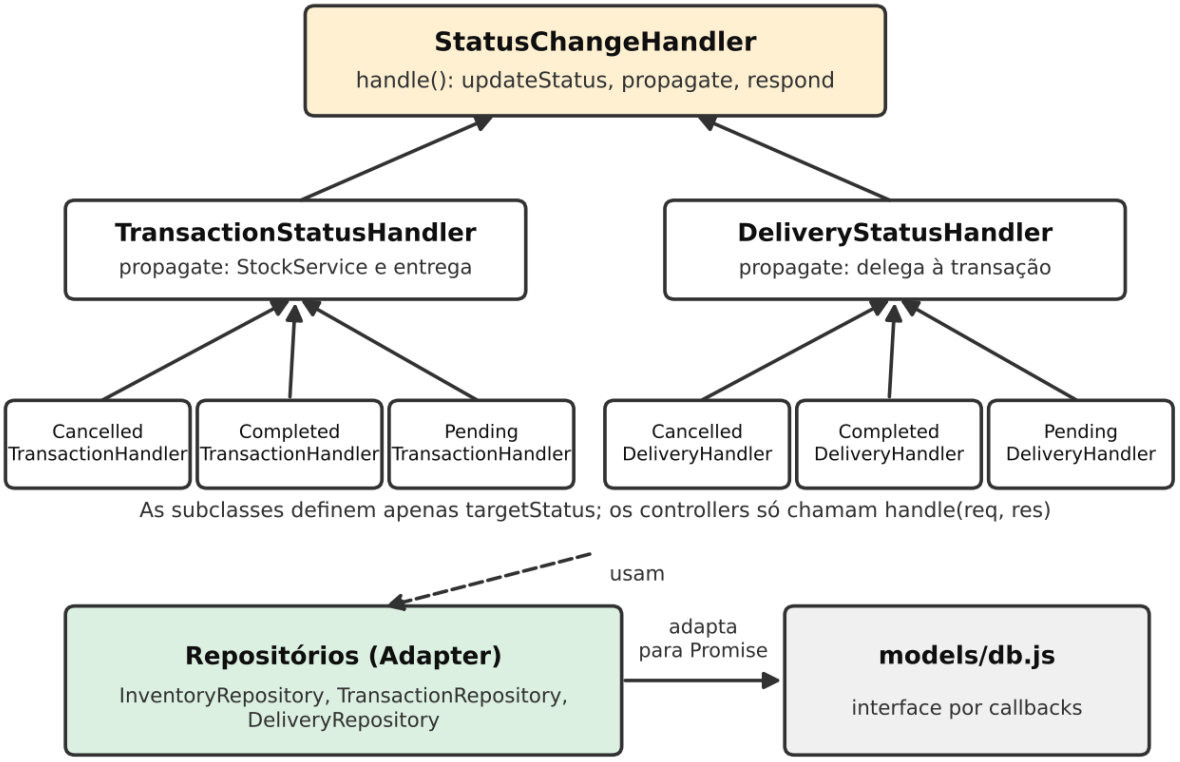


Figura 3: Template Method e Adapter (classes e relações).

4.3 Padrão 3: Template Method

- **Situação original:** seis handlers com a mesma sequência e apenas o status de destino diferente.
- **Problema:** 1088 linhas nos três handlers de transação, mais a duplicação nos handlers de entrega.
- **Solução:** a classe base define handle(req, res), que executa updateStatus, propagate e respond, nessa ordem. As subclasses por status só informam o status de destino. No fluxo da entrega, o passo propagate lê a transação e delega ao handler de transação do mesmo status, e o respond da entrega é vazio para que a resposta seja enviada uma única vez, como no original.

A Figura 6 mostra a classe base com o método handle (os três passos na ordem) e uma subclasse concreta, que só informa o status de destino. Nos controllers, postEditStatusCancelled, postEditStatusCompleted e postEditStatusPending passaram a ser apenas return cancelledHandler.handle(req, res) (e equivalentes), com os mesmos nomes exportados e a mesma assinatura.

v2-refatorado: handlers/status-change-handler.js, linhas 1 a 26

```
/* Template Method: skeleton shared by every handler that changes the status of a transaction or a delivery */

/**
 * Abstract handler. The method handle() is the template method: it fixes the order of the steps, while
 * the subclasses define what each step does.
 */
class StatusChangeHandler {
    /**
     * Status that the handler sets (abstract): 'cancelled', 'completed', or 'pending'.
     */
    get targetStatus() {
        throw new Error('targetStatus must be implemented by the subclass');
    }

    /**
     * Template method: updates the status, propagates the change, and responds to the client.
     *
     * @param {Express.Request} req  Object that contains information on the HTTP request from the client.
     * @param {Express.Response} res  Object that contains information on the HTTP response from the server.
     * @return {Promise<void>} Resolves when the response has been sent.
     */
    async handle(req, res) {
        await this.updateStatus(req);
        await this.propagate(req, res);
        this.respond(res);
    }
```

v2-refatorado: handlers/transaction-status-handlers.js, linhas 77 a 87

```
/**
 * Sets the status of a transaction to "Cancelled".
 */
class CancelledTransactionHandler extends TransactionStatusHandler {
    /**
     * @return {string} 'cancelled'.
     */
    get targetStatus() {
        return 'cancelled';
    }
}
```

Figura 6: Template Method. StatusChangeHandler e CancelledTransactionHandler (v2-refatorado).

4.4 Padrão 4: Adapter

- **Situação original:** controllers chamam db.js com callbacks aninhados e calculam o identificador da entrega somando ou subtraindo 10000000.
- **Problema:** acoplamento ao formato de callbacks e conhecimento da regra de identificadores espalhado.
- **Solução:** repositórios recebem o db no construtor e expõem métodos que retornam Promises (findById, updateStatus, findAll, saveDepleted). O módulo ids.js concentra a conversão entre identificador de transação e de entrega. db.js não foi alterado.

A Figura 7 compara o mesmo trecho do handler de entrega antes (callbacks aninhados e conta de identificador feita na mão) e depois (TransactionRepository, que devolve uma Promise sobre db.findOne). A conversão de identificadores ficou em repositories/ids.js (transactionToDelivery e deliveryToTransaction).

v1-original: controllers/delivery-controller.js, linhas 119 a 150

```
postEditStatusCompleted: function(req, res) {
    /* Retrieve the ID of the selected delivery. */
    const deliveryId = req.body.deliveryId;

    /* Set the status of the selected delivery to "Completed". */
    const update = {status: 'completed'};

    /* Use the retrieved delivery ID for database retrieval. */
    const filter = {id: deliveryId};

    /* Update the status of the delivery and its corresponding transaction. */
    db.updateOne(Delivery, filter, update, function(flag) {
        /* The transaction ID corresponding to a delivery has a starting digit of 1 and the same
         * succeeding digits as the delivery ID.
         */
        const transactionId = parseInt(deliveryId) - 10000000;

        /* Get the original status of the corresponding transaction. */
        query = {id: transactionId};
        projection = 'id status';

        db.findOne(Transaction, query, projection, function(result) {
            /* Assign the retrieved transaction details to the body of the request object
            * and set the status of the transaction to "Completed."
            */
            req.body.transactionId = result.id;
            req.body.transactionStatusOld = result.status;

            transactionController.postEditStatusCompleted(req, res);
        });
    });
},
```

v2-refatorado: repositories/transaction-repository.js, linhas 1 a 27

```
/* Adapter that exposes the transaction collection of models/db.js through a Promise-based interface */
const defaultDb = require('../models/db.js');
const Transaction = require('../models/transaction-schema.js');

/**
 * Repository for the transaction collection.
 */
class TransactionRepository {
    /**
     * @param {Object} db  Database access object (defaults to models/db.js).
     */
    constructor(db = defaultDb) {
        this.db = db;
    }

    /**
     * Retrieves a transaction using its ID.
     *
     * @param {number|string} id  ID of the transaction.
     * @param {string} projection  Fields to be returned.
     * @return {Promise<Object>} Resolves with the transaction, or false if the retrieval failed.
     */
    findById(id, projection) {
        return new Promise((resolve) => {
            this.db.findOne(Transaction, {id: id}, projection, resolve);
        });
    }
```

Figura 7: Adapter. Antes: delivery-controller.js (v1-original). Depois: transaction-repository.js (v2-refatorado).

Duas decisões de compatibilidade: as Promises dos repositórios nunca rejeitam e resolvem com o mesmo valor que o callback entregaria (findAll resolve lista vazia se o banco falhar), e saveDepleted não devolve Promise, pois o original gravava o estoque sem esperar a conclusão.

4.5 Garantia de preservação do comportamento

- **Testes de caracterização (24):** escritos antes da refatoração, sobre os handlers originais, cobrindo as transições de status, a ordem FIFO, o cálculo do identificador da entrega, o tipo de combustível desconhecido e o erro (TypeError) quando a quantidade excede o estoque. Para validar a qualidade desses testes foram introduzidas mutações no código original: 7 foram detectadas e a mutação de >= para > é um mutante equivalente, que não altera o resultado.
- **Testes de equivalência:** comparam a saída das estratégias com a do código da tag v1-original para as mesmas entradas.

- **Testes unitários novos:** estratégias, registro de combustíveis, estados, StockService, repositórios e handlers, todos com dublês, sem banco real.
- **Suíte completa:** 336 testes passando (190 originais e 146 novos), executados com Node 12.16.3.

A Figura 8 mostra a suíte original (190 testes) em v1-original e a suíte completa (336 testes) em v2-refatorado. A Figura 9 mostra os 24 testes de caracterização passando contra o código original, com o teste na versão original (sem async/await), e depois contra o código refatorado.

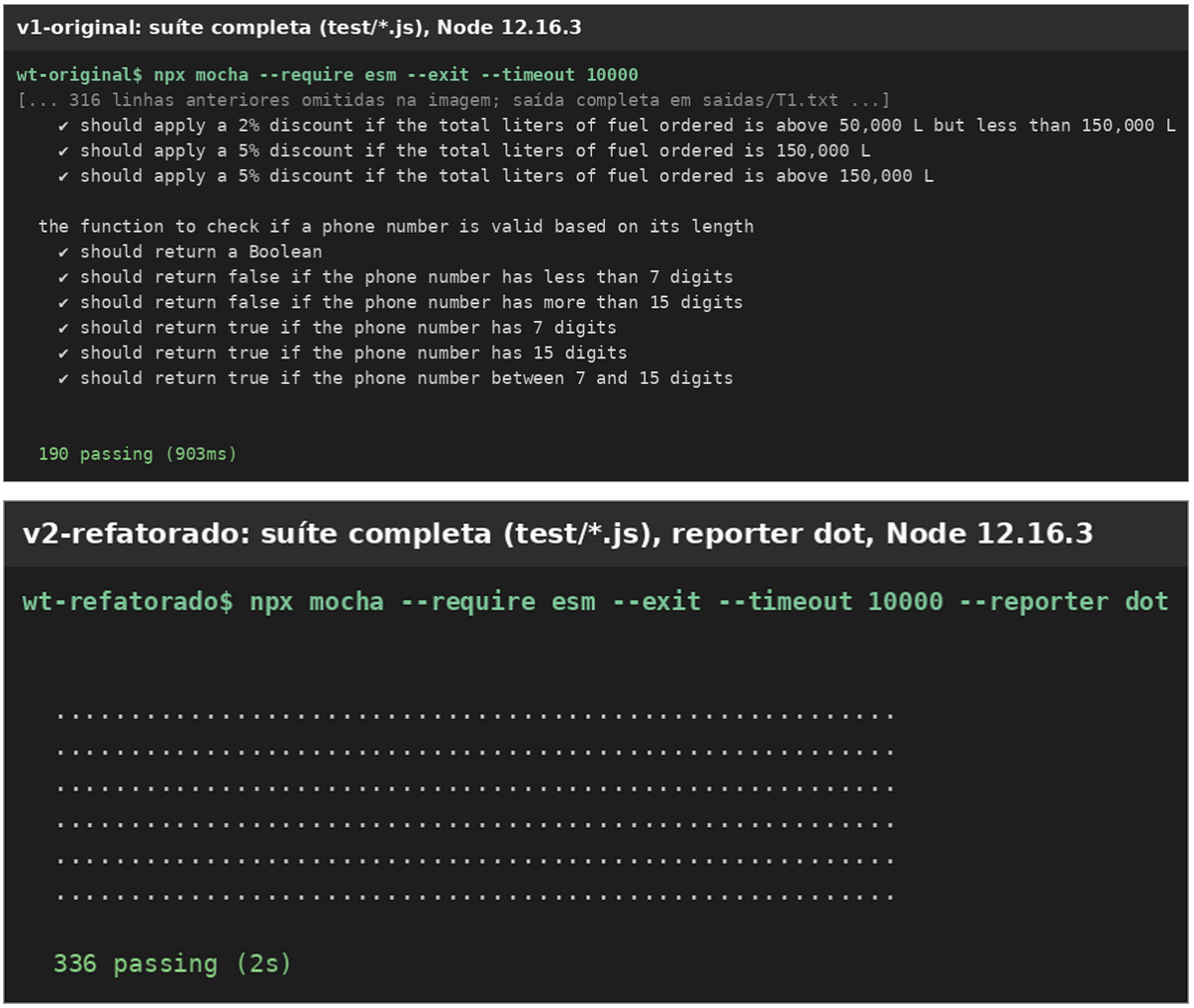


Figura 8: suíte de testes antes (190 passando) e depois (336 passando), Node 12.16.3.

v1-original + test/transaction-status-characterization-test.js da versão original (commit 5c76372, sem async/await), Node 12.16.3

```
wt-carac$ npx mocha --require esm --exit --timeout 10000 test/transaction-status-characterization-test.js


  caracterização da edição de status da transação
    postEditStatusCancelled
      ✔ pending -> cancelled devolve a um único lote parcialmente, sem tocar nos anteriores
      ✔ cancelled -> cancelled não altera o estoque, mas atualiza a entrega e responde 200
      pending -> cancelled
        ✔ devolve o estoque (FIFO) a partir do lote mais recente, para cada tipo de combustível
        ✔ atualiza a transação e a entrega (id + 10000000) para cancelled
        ✔ responde 200
      completed -> cancelled
        ✔ devolve o estoque (FIFO) a partir do lote mais recente, para cada tipo de combustível
        ✔ atualiza a transação e a entrega (id + 10000000) para cancelled
        ✔ responde 200
    postEditStatusCompleted
      ✔ cancelled -> completed consome parcialmente o lote mais antigo com saldo
      ✔ pending -> completed não altera o estoque, mas atualiza a entrega e responde 200
      ✔ completed -> completed não altera o estoque, mas atualiza a entrega e responde 200
      cancelled -> completed
        ✔ consome o estoque (FIFO) a partir do lote mais antigo, para cada tipo de combustível
        ✔ atualiza a transação e a entrega (id + 10000000) para completed
        ✔ responde 200
    postEditStatusPending
      ✔ cancelled -> pending consome parcialmente o lote mais antigo com saldo
      ✔ completed -> pending não altera o estoque, mas atualiza a entrega e responde 200
      ✔ pending -> pending não altera o estoque, mas atualiza a entrega e responde 200
      cancelled -> pending
        ✔ consome o estoque (FIFO) a partir do lote mais antigo, para cada tipo de combustível
        ✔ atualiza a transação e a entrega (id + 10000000) para pending
        ✔ responde 200
    lote de tipo desconhecido
      ✔ é tratado como kerosene: cancelled -> pending consome dele primeiro por ser o mais antigo
    transactionStatusOld indefinido
      ✔ postEditStatusCancelled não altera o estoque, mas atualiza a entrega e responde 200
      ✔ postEditStatusCompleted não altera o estoque, mas atualiza a entrega e responde 200
      ✔ postEditStatusPending não altera o estoque, mas atualiza a entrega e responde 200


  24 passing (64ms)
```

v2-refatorado: test/transaction-status-characterization-test.js (versão atual, com async/await), Node 12.16.3

```
wt-refatorado$ npx mocha --require esm --exit --timeout 10000 test/transaction-status-characterization-test.js

    [... 24 testes listados, saída idêntica à da imagem anterior ...]

  24 passing (57ms)
```

Figura 9: testes de caracterização contra v1-original (cima) e contra v2-refatorado (baixo).

A Figura 10 mostra a única alteração feita em teste existente: 23 linhas trocadas por async/await. Ao remover async/await do arquivo atual, a diferença para o original é vazia (exit 0), ou seja, nenhum valor esperado foi alterado.

Diff de test/transaction-status-characterization-test.js: 5c76372 (original) vs v2-refatorado

```
powerzone-inventory$ git diff --stat 5c76372 v2-refatorado -- test/transaction-status-characterization-test.js
 test/transaction-status-characterization-test.js | 46 ++++++++++++-----------
 1 file changed, 23 insertions(+), 23 deletions(-)

powerzone-inventory$ git diff 5c76372 v2-refatorado -- test/transaction-status-characterization-test.js
diff --git a/test/transaction-status-characterization-test.js b/test/transaction-status-characterization-test.js
index b5df521..6909260 100644
--- a/test/transaction-status-characterization-test.js
+++ b/test/transaction-status-characterization-test.js
@@ -150,7 +150,7 @@ describe('caracterização da edição de status da transação', function() {
         initial = initialDepletion(stockList);
     }

-   beforeEach(function() {
+   beforeEach(async function() {
         res = {
```

test/transaction-status-characterization-test.js: original vs v2-refatorado sem "async"/"await"

```
powerzone-inventory$ diff <(git show 5c76372:test/transaction-status-characterization-test.js) \
>   <(git show v2-refatorado:test/transaction-status-characterization-test.js | sed -E 's/async function/function/g; s/await //g'); \
>   echo "diff exit $?"
diff exit 0
```

Figura 10: diff do teste de caracterização entre o commit original (5c76372) e v2-refatorado.

5. Análise dos Resultados

Comparação entre a tag v1-original e a tag v2-refatorado:

Métrica	Antes	Depois
Linhas em transaction-controller.js	2804	1753
Ocorrências de if em transaction-controller.js	234	144
Linhas em delivery-controller.js	470	397
Laços while, ordenações e gravações de estoque duplicados	15 de cada	1 de cada (nas estratégias)
Arquivos de lógica extraída (services, repositories, handlers)	0	235, 143 e 309 linhas
Testes passando	190	336

Os números de if e de linhas caem porque a lógica dos seis handlers foi movida para classes pequenas. O diff total contra v1-original é de 21 arquivos, com 2177 linhas adicionadas e 1159 removidas, e o acréscimo se deve em grande parte aos novos arquivos de teste.

- **Acoplamento:** os controllers não acessam mais db.js nos handlers de status. Eles dependem apenas dos handlers, que recebem repositórios e serviço por injeção no construtor.
- **Coesão:** cada classe tem uma responsabilidade (aplicar uma política de estoque, representar um estado, atualizar um registro).
- **Duplicação:** a regra FIFO passou de 15 cópias para uma por política, e os seis handlers viraram subclasses que definem apenas o status de destino.
- **Manutenção e extensibilidade:** um novo combustível exige uma entrada no registro fuel-types.js. Um novo status exige uma nova subclasse de estado e de handler. Antes, ambos exigiam editar os seis handlers.
- **Reutilização:** estratégias, estados e repositórios podem ser usados por outros fluxos, e os testes com dublês dispensam o banco.
- **Legibilidade:** os handlers ficaram com uma ou duas linhas por método, e a regra de negócio pode ser lida em services.

6. Conclusão

A refatoração substituiu seis handlers longos e duplicados por uma composição de quatro padrões: Adapter isolou o acesso aos dados, State e Strategy concentraram a regra de estoque e Template Method uniformizou o fluxo de mudança de status. O comportamento observável foi mantido, o que foi verificado pelos 24 testes de caracterização, pelos testes de equivalência e pela suíte completa de 336 testes.

Os padrões contribuíram para o reuso porque separaram as partes que variam (status e política de estoque) das que se repetem (sequência de passos e acesso ao banco), permitindo estender o sistema com poucas alterações.

Dificuldades e limitações:

- Escrever testes de caracterização antes de refatorar exigiu entender cada ramo do código original, e validá-los por mutação mostrou que alguns casos não eram detectados.
- Os handlers passaram a ser assíncronos, o que exigiu o ajuste mecânico (async/await) nos testes de caracterização.
- Comportamentos frágeis do original foram preservados de propósito: o TypeError quando a quantidade excede o estoque não é tratado e a resposta não é enviada; a gravação do estoque não é aguardada.
- A atualização da entrega no fluxo da entrega continua sendo feita duas vezes, para manter o comportamento.
- Os métodos postEditTransaction e postAddTransaction ainda têm lógica de estoque duplicada e ficaram como trabalho futuro.
- O lint do projeto aponta erros em services e em alguns testes (por exemplo, falta de JSDoc). Os arquivos de handlers e repositórios estão limpos.

Aprendizados da equipe: refatorar com testes de caracterização reduz o risco; aplicar padrões em conjunto produz resultado melhor do que aplicá-los isoladamente; e a divisão do trabalho em branches e Pull Requests facilitou a revisão e a integração.

7. Referências

- GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos**. Porto Alegre: Bookman, 2000.
- FOWLER, M. **Refatoração: Aperfeiçoando o Projeto de Código Existente**. 2. ed. São Paulo: Novatec, 2020.
- FEATHERS, M. **Working Effectively with Legacy Code**. Prentice Hall, 2004.
- REFACTORING GURU. **Design Patterns**: Strategy, State, Template Method e Adapter. Disponível em: https://refactoring.guru/design-patterns.